简单学 Zig

Zig 编程语言的简单实用指南

目录

- 0. Hello World
- 1. Zig 简介
- 2. 变量和类型
- 3. 控制流
- 4. 函数
- 5. 数组和切片
- 6. 结构体
- 7. 枚举和联合体
- 8. 错误处理
- 9. 可选值
- 10. 指针
- 11. 内存分配
- 12. 编译时执行
- 13. 测试
- 14. 构建系统

---

0. Hello World

让我们从最简单的 Zig 程序开始，然后看看内存分配。

如何运行

将代码保存到文件（例如 `hello.zig`），然后用以下命令运行：

```
zig run hello.zig
```

就这么简单！Zig 一条命令就能编译并运行你的程序。

其他构建方式

```
┌──────────────────────────────────────┐
│  zig run file.zig      # 编译后运行     │
│  zig build-exe file.zig # 仅编译       │
│  ./file                 # 运行二进制文件 │
└──────────────────────────────────────┘
```

```
运行 Zig 程序:

┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│  hello.zig   │ -> │   zig run    │ -> │    输出      │
│  (源码)      │   │  (编译)      │   │  (结果)      │
└──────────────┘   └──────────────┘   └──────────────┘

使用 defer 的内存管理:
```

```
┌──────────────────────────────┐
│  const x = try allocate();   │
│  defer free(x);  <- 已注册    │
│                              │
│  ... 使用 x ...              │
│                              │
└──────────────────────────────┘
          │
          ▼ (作用域结束)
┌──────────────────────────────┐
│  free(x) <- 执行了!           │
└──────────────────────────────┘
```

示例: **hello_world**

最简单的 Zig 程序 - 在控制台输出 Hello World

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("Hello, World!\n", .{});
}
```

输出:

```
Hello, World!
```

示例: **allocator_example**

使用 defer 自动清理和内存分配的综合示例

```zig
const std = @import("std");

// Function that takes an allocator and creates a dynamic string
fn createGreeting(allocator: std.mem.Allocator, name: []const u8) ![]u8 {
    // Allocate memory for the greeting
    const greeting = try std.fmt.allocPrint(allocator, "Hello, {s}! Welcome to Zig.", .{name});
    return greeting;
}

// Function that creates a dynamic array
fn createNumbers(allocator: std.mem.Allocator, count: usize) ![]u32 {
    // Allocate an array of u32
    const numbers = try allocator.alloc(u32, count);

    // Fill with values
    for (numbers, 0..) |*num, i| {
        num.* = @intCast(i * 10);
    }

    return numbers;
}

pub fn main() !void {
    // Create a General Purpose Allocator (GPA)
```

```zig
    // GPA detects memory leaks and double-frees in debug mode
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer {
        const status = gpa.deinit();
        if (status == .leak) {
            std.debug.print("Memory leak detected!\n", .{});
        }
    }

    const allocator = gpa.allocator();

    // Example 1: Using defer for automatic cleanup
    std.debug.print("=== Example 1: String with defer ===\n", .{});
    {
        const greeting = try createGreeting(allocator, "Alice");
        defer allocator.free(greeting); // Automatically freed when scope ends

        std.debug.print("{s}\n", .{greeting});
    } // greeting is freed here

    // Example 2: Dynamic array with defer
    std.debug.print("\n=== Example 2: Array with defer ===\n", .{});
    {
        const numbers = try createNumbers(allocator, 5);
        defer allocator.free(numbers); // Automatically freed when scope ends

        std.debug.print("Numbers: ", .{});
        for (numbers) |n| {
            std.debug.print("{d} ", .{n});
        }
        std.debug.print("\n", .{});
    } // numbers is freed here

    // Example 3: Multiple allocations with defer (LIFO order)
    std.debug.print("\n=== Example 3: Multiple defers (LIFO order) ===\n", .{});
    {
        const str1 = try createGreeting(allocator, "Bob");
        defer {
            std.debug.print("Freeing str1\n", .{});
            allocator.free(str1);
        }

        const str2 = try createGreeting(allocator, "Charlie");
        defer {
            std.debug.print("Freeing str2\n", .{});
            allocator.free(str2);
        }

        std.debug.print("{s}\n", .{str1});
        std.debug.print("{s}\n", .{str2});
    } // str2 freed first, then str1 (LIFO)

    // Example 4: Using ArrayListUnmanaged with allocator
```

```zig
    std.debug.print("\n=== Example 4: ArrayListUnmanaged ===\n", .{});
    {
        var list: std.ArrayListUnmanaged(i32) = .{};
        defer list.deinit(allocator); // Pass allocator to deinit

        try list.append(allocator, 100);
        try list.append(allocator, 200);
        try list.append(allocator, 300);

        std.debug.print("ArrayList items: ", .{});
        for (list.items) |item| {
            std.debug.print("{d} ", .{item});
        }
        std.debug.print("\n", .{});
    }

    std.debug.print("\n=== All memory properly freed! ===\n", .{});
}
```

输出:

```
=== Example 1: String with defer ===
Hello, Alice! Welcome to Zig.

=== Example 2: Array with defer ===
Numbers: 0 10 20 30 40

=== Example 3: Multiple defers (LIFO order) ===
Hello, Bob! Welcome to Zig.
Hello, Charlie! Welcome to Zig.
Freeing str2
Freeing str1

=== Example 4: ArrayListUnmanaged ===
ArrayList items: 100 200 300

=== All memory properly freed! ===
```

---

1. Zig 简介

Zig 是一种为以下目的设计的现代系统编程语言：- 性能: 没有隐藏的控制流或分配 - 安全性: 可选的安全检查，没有未定义行为 - 简洁性: 没有隐藏的魔法，代码易读 - 互操作性: 直接兼容 C ABI

开始使用

从 https://ziglang.org/download/ 安装 Zig

```
┌──────────────────────────────────────┐
│            Zig 工作流程              │
├──────────────────────────────────────┤
│  1. 写代码     →  main.zig           │
│  2. 编译       →  zig build-exe      │
```

```
│  3. 运行      →  ./main                  │
│                                           │
│  或者简单点:  →  zig run main.zig         │
└──────────────────────────────────────────┘
```

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│    源码      │ -> │   编译器     │ -> │   二进制     │
│  main.zig    │   │    zig       │   │   ./main     │
└──────────────┘   └──────────────┘   └──────────────┘
```

示例: **hello_world**

经典的 Hello World 程序

```zig
const std = @import("std");

// This is your first Zig program!
// std.debug.print outputs to stderr
pub fn main() void {
    std.debug.print("Hello, Zig!\n", .{});
}
```

输出:

```
Hello, Zig!
```

---

2. 变量和类型

Zig 有一个带显式类型的简单类型系统。

变量声明

```
┌──────────────────────────────────────────┐
│  const x = 5;     // 不可变 (const)      │
│  var y = 10;      // 可变 (var)          │
│  var z: i32 = 0; // 显式类型             │
└──────────────────────────────────────────┘
```

基本类型

类型	描述	大小
i8-i128	有符号整数	1-16 B
u8-u128	无符号整数	1-16 B
f32,f64	浮点数	4,8 B
bool	布尔值	1 B
void	无值	0 B

```
内存布局(小端序):

u8:   [xxxxxxxx]           (1 字节)
u16:  [xxxxxxxx][xxxxxxxx] (2 字节)
```

```
u32:  [xxxx][xxxx][xxxx][xxxx] (4 字节)

i8 范围:  -128 到 127
u8 范围:  0 到 255
```

示例: **variables**

变量声明和基本类型

```zig
const std = @import("std");

pub fn main() void {
    // Immutable constant
    const message = "Hello";

    // Mutable variable
    var count: i32 = 0;
    count += 1;

    // Type inference
    const pi = 3.14159;

    // Explicit type
    const byte: u8 = 255;

    std.debug.print("message: {s}\n", .{message});
    std.debug.print("count: {}\n", .{count});
    std.debug.print("pi: {d:.2}\n", .{pi});
    std.debug.print("byte: {}\n", .{byte});
}
```

输出:

```
message: Hello
count: 1
pi: 3.14
byte: 255
```

示例: **type_coercion**

类型转换和强制转换

```zig
const std = @import("std");

pub fn main() void {
    // Integer types
    const small: u8 = 10;
    const big: u32 = small; // Safe widening

    // Explicit cast for narrowing
    const large: u32 = 1000;
    const truncated: u8 = @truncate(large);

    std.debug.print("small: {}, big: {}\n", .{ small, big });
    std.debug.print("large: {}, truncated: {}\n", .{ large, truncated });
}
```

输出:

```
small: 10, big: 10
large: 1000, truncated: 232
```

---

3. 控制流

Zig 提供标准控制流，还有一些独特的功能。

If/Else

```
┌──────────────────────────────────────┐
│  if (条件) {                          │
│      // 真分支                        │
│  } else {                             │
│      // 假分支                        │
│  }                                    │
└──────────────────────────────────────┘
```

While 循环

```
┌──────────────────────────────────────┐
│  while (条件) : (继续表达式) {          │
│      // 循环体                        │
│  }                                    │
└──────────────────────────────────────┘
```

控制流图:

```
    ┌─────────┐
    │ 开始    │
    └─────────┘
        │
    ┌───▼─────┐        是        ┌─────────┐
    │ 条件?   │──────────────────►│ 代码块  │
    └─────────┘                  └─────────┘
        │ 否
    ┌───▼─────┐
    │ 结束    │
    └─────────┘
```

示例: **if_else**

If/else 语句和表达式

```zig
const std = @import("std");

pub fn main() void {
    const x: i32 = 42;

    // Basic if/else
    if (x > 0) {
```

```
        std.debug.print("x is positive\n", .{});
    } else if (x < 0) {
        std.debug.print("x is negative\n", .{});
    } else {
        std.debug.print("x is zero\n", .{});
    }

    // If as expression
    const abs_x = if (x < 0) -x else x;
    std.debug.print("abs(x) = {}\n", .{abs_x});
}
```

输出:

```
x is positive
abs(x) = 42
```

示例: **loops**

While 和 for 循环

```
const std = @import("std");

pub fn main() void {
    // While loop
    var i: u32 = 0;
    while (i < 3) : (i += 1) {
        std.debug.print("while i = {}\n", .{i});
    }

    // For loop over range
    for (0..3) |j| {
        std.debug.print("for j = {}\n", .{j});
    }

    // For loop over array
    const arr = [_]i32{ 10, 20, 30 };
    for (arr) |val| {
        std.debug.print("val = {}\n", .{val});
    }
}
```

输出:

```
while i = 0
while i = 1
while i = 2
for j = 0
for j = 1
for j = 2
val = 10
val = 20
val = 30
```

示例: **switch**

Switch 表达式

```zig
const std = @import("std");

pub fn main() void {
    const day: u8 = 3;

    // Switch statement
    const name = switch (day) {
        1 => "Monday",
        2 => "Tuesday",
        3 => "Wednesday",
        4 => "Thursday",
        5 => "Friday",
        6, 7 => "Weekend",
        else => "Invalid",
    };

    std.debug.print("Day {}: {s}\n", .{ day, name });
}
```

输出:

```
Day 3: Wednesday
```

---

4. 函数

函数在 Zig 中是一等公民。

函数语法

```
┌──────────────────────────────────────┐
│  fn 名称(参数: 类型) 返回类型 {        │
│      return 值;                      │
│  }                                   │
└──────────────────────────────────────┘
```

特殊功能

- 函数可以作为值传递
- 编译时函数求值
- 使用 inline 的内联函数

函数调用栈:

```
┌──────────────────┐
│  main()          │  <- 栈帧
├──────────────────┤
│  add(3, 4)       │  <- 新帧
│  a = 3, b = 4    │
│  return 7        │
```

```
├──────────────────────┤
│   (返回到 main)      │
└──────────────────────┘
```

示例: **basic_functions**

基本函数定义

```zig
const std = @import("std");

// Simple function
fn add(a: i32, b: i32) i32 {
    return a + b;
}

// Function with multiple return (tuple)
fn divmod(a: i32, b: i32) struct { quot: i32, rem: i32 } {
    return .{ .quot = @divTrunc(a, b), .rem = @mod(a, b) };
}

pub fn main() void {
    const sum = add(3, 4);
    std.debug.print("3 + 4 = {}\n", .{sum});

    const result = divmod(17, 5);
    std.debug.print("17 / 5 = {} remainder {}\n", .{ result.quot, result.rem });
}
```

输出:

```
3 + 4 = 7
17 / 5 = 3 remainder 2
```

示例: **function_pointers**

函数指针和高阶函数

```zig
const std = @import("std");

fn double(x: i32) i32 {
    return x * 2;
}

fn triple(x: i32) i32 {
    return x * 3;
}

// Function that takes a function pointer
fn apply(value: i32, func: *const fn (i32) i32) i32 {
    return func(value);
}

pub fn main() void {
    const x: i32 = 5;
```

```
    std.debug.print("double({}) = {}\n", .{ x, apply(x, &double) });
    std.debug.print("triple({}) = {}\n", .{ x, apply(x, &triple) });
}
```

输出:

```
double(5) = 10
triple(5) = 15
```

---

5. 数组和切片

数组有固定大小；切片是数组的视图。

数组 vs 切片

```
┌──────────────────────────────────────┐
│  数组: [N]T    - 编译时固定大小        │
│  切片: []T     - 运行时数组视图        │
└──────────────────────────────────────┘
```

```
内存中的数组:

┌───┬───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │ 5 │   arr[5]i32
└───┴───┴───┴───┴───┘
  0   1   2   3   4

切片（视图）:

        ┌──────────┐
        │ ptr + len │
        └──────────┘
              │
              ▼
┌───┬───┬───┬───┬───┐
│ 1 │ 2 │ 3 │ 4 │ 5 │
└───┴───┴───┴───┴───┘
        └───────┘
        slice [1..4]
```

示例: arrays

数组基础

```zig
const std = @import("std");

pub fn main() void {
    // Array with explicit size
    const arr1: [5]i32 = [_]i32{ 1, 2, 3, 4, 5 };

    // Array with inferred size
    const arr2 = [_]i32{ 10, 20, 30 };

    // Access elements
    std.debug.print("arr1[0] = {}\n", .{arr1[0]});
```

```
    std.debug.print("arr2.len = {}\n", .{arr2.len});

    // Iterate
    var sum: i32 = 0;
    for (arr1) |x| {
        sum += x;
    }
    std.debug.print("sum of arr1 = {}\n", .{sum});
}
```

输出:

```
arr1[0] = 1
arr2.len = 3
sum of arr1 = 15
```

示例: slices

切片 - 数组的视图

```
const std = @import("std");

pub fn main() void {
    const arr = [_]i32{ 1, 2, 3, 4, 5 };

    // Create slice from array
    const slice: []const i32 = arr[1..4]; // [2, 3, 4]

    std.debug.print("slice.len = {}\n", .{slice.len});

    for (slice, 0..) |val, i| {
        std.debug.print("slice[{}] = {}\n", .{ i, val });
    }
}
```

输出:

```
slice.len = 3
slice[0] = 2
slice[1] = 3
slice[2] = 4
```

示例: string_literals

字符串作为字节切片

```
const std = @import("std");

pub fn main() void {
    // String literal is []const u8
    const hello = "Hello, World!";

    std.debug.print("String: {s}\n", .{hello});
    std.debug.print("Length: {}\n", .{hello.len});
    std.debug.print("First char: {c}\n", .{hello[0]});
```

```zig
    // Iterate over characters
    for (hello[0..5]) |c| {
        std.debug.print("{c} ", .{c});
    }
    std.debug.print("\n", .{});
}
```

输出:

```
String: Hello, World!
Length: 13
First char: H
H e l l o
```

---

6. 结构体

结构体将相关数据组合在一起。

结构体定义

```
┌──────────────────────────────────────┐
│  const Point = struct {              │
│      x: i32,                         │
│      y: i32,                         │
│                                      │
│      fn init(x: i32, y: i32) Point {  │
│          return .{ .x = x, .y = y };  │
│      }                               │
│  };                                  │
└──────────────────────────────────────┘
```

结构体内存布局:

```
Point { x: i32, y: i32 }

┌──────────────┬──────────────┐
│   x: i32     │   y: i32     │
│   4 字节     │   4 字节     │
└──────────────┴──────────────┘
     总共: 8 字节
```

示例: **basic_struct**

带方法的结构体定义

```zig
const std = @import("std");

// Define a struct
const Point = struct {
    x: i32,
    y: i32,
```

```zig
    // Method
    fn distance_from_origin(self: Point) f32 {
        const fx: f32 = @floatFromInt(self.x);
        const fy: f32 = @floatFromInt(self.y);
        return @sqrt(fx * fx + fy * fy);
    }
};

pub fn main() void {
    // Create instance
    const p = Point{ .x = 3, .y = 4 };

    std.debug.print("Point: ({}, {})\n", .{ p.x, p.y });
    std.debug.print("Distance: {d:.2}\n", .{p.distance_from_origin()});
}
```

输出:

```
Point: (3, 4)
Distance: 5.00
```

示例: **struct_defaults**

带默认字段值的结构体

```zig
const std = @import("std");

const Config = struct {
    // Fields with default values
    width: u32 = 800,
    height: u32 = 600,
    title: []const u8 = "Untitled",
};

pub fn main() void {
    // Use all defaults
    const c1 = Config{};

    // Override some defaults
    const c2 = Config{ .width = 1920, .height = 1080 };

    std.debug.print("c1: {}x{} '{s}'\n", .{ c1.width, c1.height, c1.title });
    std.debug.print("c2: {}x{} '{s}'\n", .{ c2.width, c2.height, c2.title });
}
```

输出:

```
c1: 800x600 'Untitled'
c2: 1920x1080 'Untitled'
```

---

7. 枚举和联合体

枚举定义一组命名值。标记联合体将枚举与数据结合。

枚举

```
┌──────────────────────────────────────┐
│  const Color = enum { red, green, blue };│
└──────────────────────────────────────┘
```

标记联合体

```
┌──────────────────────────────────────┐
│  const Value = union(enum) {          │
│      int: i32,                        │
│      float: f64,                      │
│      none,                            │
│  };                                   │
└──────────────────────────────────────┘
```

标记联合体内存:

Value union(enum):

```
┌──────────┬──────────────┐
│   标记   │     数据     │
│ (enum)   │ (最大的字段) │
└──────────┴──────────────┘
```

标记值: .int=0, .float=1, .boolean=2, .none=3

示例: **enums**

带方法的枚举

```zig
const std = @import("std");

const Direction = enum {
    north,
    south,
    east,
    west,

    // Enum method
    fn opposite(self: Direction) Direction {
        return switch (self) {
            .north => .south,
            .south => .north,
            .east => .west,
            .west => .east,
        };
    }
};

pub fn main() void {
    const dir = Direction.north;
    const opp = dir.opposite();

    std.debug.print("Direction: {}\n", .{dir});
```

```
    std.debug.print("Opposite: {}\n", .{opp});
}
```

输出:

```
Direction: .north
Opposite: .south
```

示例: **tagged_union**

类型安全变体的标记联合体

```
const std = @import("std");

const Value = union(enum) {
    int: i32,
    float: f64,
    boolean: bool,
    none: void,
};

fn printValue(val: Value) void {
    switch (val) {
        .int => |i| std.debug.print("int: {}\n", .{i}),
        .float => |f| std.debug.print("float: {d:.2}\n", .{f}),
        .boolean => |b| std.debug.print("bool: {}\n", .{b}),
        .none => std.debug.print("none\n", .{}),
    }
}

pub fn main() void {
    const v1 = Value{ .int = 42 };
    const v2 = Value{ .float = 3.14 };
    const v3 = Value{ .boolean = true };
    const v4 = Value{ .none = {} };

    printValue(v1);
    printValue(v2);
    printValue(v3);
    printValue(v4);
}
```

输出:

```
int: 42
float: 3.14
bool: true
none
```

---

8. 错误处理

Zig 使用错误联合体进行显式错误处理。

错误联合体类型

```
┌──────────────────────────────────────┐
│  fn divide(a: i32, b: i32) !i32 {    │
│      if (b == 0) return error.DivByZero;│
│      return @divTrunc(a, b);         │
│  }                                   │
└──────────────────────────────────────┘
```

错误处理选项

- `try`: 将错误向上传播
- `catch`: 处理错误
- `orelse`: 错误时的默认值

错误联合体类型:

```
    anyerror!T 或 ErrorSet!T
          │
     ┌────┴────┐
     │         │
  错误        值
     │         │
┌────▼────┐ ┌────▼────┐
│ catch   │ │ 成功    │
└─────────┘ └─────────┘
```

示例: **error_basics**

使用 catch 的基本错误处理

```zig
const std = @import("std");

const MathError = error{
    DivisionByZero,
    Overflow,
};

fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return MathError.DivisionByZero;
    return @divTrunc(a, b);
}

pub fn main() void {
    // Using catch to handle error
    const result1 = divide(10, 2) catch |err| {
        std.debug.print("Error: {}\n", .{err});
        return;
    };
    std.debug.print("10 / 2 = {}\n", .{result1});

    // Using catch with default
    const result2 = divide(10, 0) catch 0;
    std.debug.print("10 / 0 = {} (default)\n", .{result2});
```

```
}
```

输出:

```
10 / 2 = 5
10 / 0 = 0 (default)
```

示例: **try_keyword**

使用 try 传播错误

```zig
const std = @import("std");

const ParseError = error{InvalidChar};

fn parseDigit(c: u8) ParseError!u8 {
    if (c >= '0' and c <= '9') {
        return c - '0';
    }
    return ParseError.InvalidChar;
}

fn parseNumber(s: []const u8) ParseError!u32 {
    var result: u32 = 0;
    for (s) |c| {
        // try propagates error if parseDigit fails
        const digit = try parseDigit(c);
        result = result * 10 + digit;
    }
    return result;
}

pub fn main() void {
    if (parseNumber("123")) |num| {
        std.debug.print("Parsed: {}\n", .{num});
    } else |err| {
        std.debug.print("Parse error: {}\n", .{err});
    }

    if (parseNumber("12x")) |num| {
        std.debug.print("Parsed: {}\n", .{num});
    } else |err| {
        std.debug.print("Parse error: {}\n", .{err});
    }
}
```

输出:

```
Parsed: 123
Parse error: error.InvalidChar
```

---

9. 可选值

可选值表示可能存在或不存在的值。

可选类型

```
┌──────────────────────────────────────┐
│  var x: ?i32 = null;   // 无值        │
│  x = 42;               // 有值        │
│                                       │
│  if (x) |val| {        // 解包        │
│      // 使用 val                      │
│  }                                    │
└──────────────────────────────────────┘
```

可选类型 ?T:

```
┌──────────────┐
│   ?i32       │
└──────────────┘
       │
   ┌───┴───┐
   │       │
 null     值
   │       │
┌──▼──┐  ┌──▼──┐
│ 空  │  │ 42  │
└─────┘  └─────┘
```

示例: optionals

可选类型和解包

```zig
const std = @import("std");

fn find_first_even(arr: []const i32) ?i32 {
    for (arr) |x| {
        if (@mod(x, 2) == 0) return x;
    }
    return null; // Not found
}

pub fn main() void {
    const arr1 = [_]i32{ 1, 3, 4, 7, 9 };
    const arr2 = [_]i32{ 1, 3, 5, 7, 9 };

    // Using if to unwrap
    if (find_first_even(&arr1)) |val| {
        std.debug.print("First even: {}\n", .{val});
    } else {
        std.debug.print("No even number found\n", .{});
    }

    // Using orelse for default
    const result = find_first_even(&arr2) orelse -1;
    std.debug.print("Result with default: {}\n", .{result});
}
```

输出:

```
First even: 4
Result with default: -1
```

示例: **optional_pointers**

用于链接结构的可选指针

```zig
const std = @import("std");

const Node = struct {
    value: i32,
    next: ?*Node, // Optional pointer
};

pub fn main() void {
    var node3 = Node{ .value = 30, .next = null };
    var node2 = Node{ .value = 20, .next = &node3 };
    var node1 = Node{ .value = 10, .next = &node2 };

    // Traverse linked list
    var current: ?*Node = &node1;
    while (current) |node| {
        std.debug.print("Value: {}\n", .{node.value});
        current = node.next;
    }
}
```

输出:

```
Value: 10
Value: 20
Value: 30
```

---

10. 指针

Zig 有不同的指针类型用于不同的用例。

指针类型

```
┌──────────────────────────────────────┐
│  *T        - 单项指针                │
│  [*]T      - 多项指针                │
│  *const T - 指向 const 的指针        │
│  ?*T       - 可选指针                │
└──────────────────────────────────────┘
```

指向变量的指针:

```
   ptr          x
┌─────────┐  ┌─────────┐
│ 地址 ───┼──►│   10    │
└─────────┘  └─────────┘
```

```
*ptr 解引用获取值
&x 获取 x 的地址
```

示例: **pointers_basic**

基本指针操作

```zig
const std = @import("std");

pub fn main() void {
    var x: i32 = 10;

    // Get pointer
    const ptr: *i32 = &x;

    // Dereference
    std.debug.print("x = {}\n", .{x});
    std.debug.print("*ptr = {}\n", .{ptr.*});

    // Modify through pointer
    ptr.* = 20;
    std.debug.print("After modification: x = {}\n", .{x});
}
```

输出:

```
x = 10
*ptr = 10
After modification: x = 20
```

示例: **pointer_arithmetic**

指针算术和切片

```zig
const std = @import("std");

pub fn main() void {
    const arr = [_]i32{ 10, 20, 30, 40, 50 };

    // Many-item pointer
    const ptr: [*]const i32 = &arr;

    // Access by index
    std.debug.print("ptr[0] = {}\n", .{ptr[0]});
    std.debug.print("ptr[2] = {}\n", .{ptr[2]});

    // Slice from pointer
    const slice = ptr[1..4];
    for (slice) |val| {
        std.debug.print("slice val: {}\n", .{val});
    }
}
```

输出:

```
ptr[0] = 10
ptr[2] = 30
slice val: 20
slice val: 30
slice val: 40
```

---

11. 内存分配

Zig 让你对内存分配有显式控制。

分配器接口

```
┌──────────────────────────────────────────┐
│  const allocator = std.heap.page_allocator;│
│  const ptr = try allocator.create(T);    │
│  defer allocator.destroy(ptr);           │
└──────────────────────────────────────────┘
```

常见分配器

- page_allocator: 操作系统页面分配
- GeneralPurposeAllocator: 调试友好
- ArenaAllocator: 批量释放

```
内存管理:

栈(自动):
┌──────────────┐
│  局部变量     │ <- 快速,自动清理
└──────────────┘

堆(手动):
┌──────────────┐
│ allocator.alloc │ <- 显式分配
│ allocator.free  │ <- 显式释放
└──────────────┘

用 defer 清理!
```

示例: **allocation**

动态内存分配

```zig
const std = @import("std");

pub fn main() !void {
    // Use general purpose allocator
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // Allocate array
```

```zig
    const arr = try allocator.alloc(i32, 5);
    defer allocator.free(arr);

    // Initialize
    for (arr, 0..) |*item, i| {
        item.* = @intCast(i * 10);
    }

    // Print
    for (arr) |val| {
        std.debug.print("{} ", .{val});
    }
    std.debug.print("\n", .{});
}
```

输出:

```
0 10 20 30 40
```

示例: **arraylist**

使用 ArrayList 做动态数组

```zig
const std = @import("std");

pub fn main() !void {
    var gpa: std.heap.GeneralPurposeAllocator(.{}) = .init;
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // ArrayList - dynamic array
    var list: std.ArrayListUnmanaged(i32) = .empty;
    defer list.deinit(allocator);

    // Add elements
    try list.append(allocator, 10);
    try list.append(allocator, 20);
    try list.append(allocator, 30);

    std.debug.print("List: ", .{});
    for (list.items) |val| {
        std.debug.print("{} ", .{val});
    }
    std.debug.print("\n", .{});
    std.debug.print("Length: {}\n", .{list.items.len});
}
```

输出:

```
List: 10 20 30
Length: 3
```

---

12. 编译时执行

Zig 可以使用 `comptime` 在编译时执行代码。

Comptime 功能

```
┌──────────────────────────────────────┐
│  comptime {                          │
│      // 在编译时运行                  │
│  }                                   │
│                                      │
│  fn generic(comptime T: type) type {  │
│      // 类型级别计算                  │
│  }                                   │
└──────────────────────────────────────┘
```

编译时 vs 运行时:

```
┌──────────────────┐        ┌──────────────────┐
│     编译时        │        │     运行时        │
├──────────────────┤        ├──────────────────┤
│ comptime 块       │        │ 普通代码          │
│ 类型计算          │        │ 用户输入          │
│ 泛型参数          │        │ 动态数据          │
│ 常量折叠          │        │ 堆分配            │
└──────────────────┘        └──────────────────┘
          │                          │
          └──────────────────────────┘
                       │
                  最终二进制
```

示例: **comptime_basic**

编译时计算

```zig
const std = @import("std");

// Compile-time function
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// Const values with comptime-known args are computed at compile time
const factorial_10 = factorial(10);

pub fn main() void {
    // This value is computed at compile time!
    std.debug.print("10! = {}\n", .{factorial_10});

    // Comptime type introspection
    const T = i32;
    std.debug.print("Size of i32: {} bytes\n", .{@sizeOf(T)});
```

```zig
    // Inline comptime block
    const squares = comptime blk: {
        var arr: [5]i32 = undefined;
        for (0..5) |i| {
            arr[i] = @as(i32, @intCast(i)) * @as(i32, @intCast(i));
        }
        break :blk arr;
    };
    std.debug.print("Squares: ", .{});
    for (squares) |s| {
        std.debug.print("{} ", .{s});
    }
    std.debug.print("\n", .{});
}
```

输出:

```
10! = 3628800
Size of i32: 4 bytes
Squares: 0 1 4 9 16
```

示例: **generic_function**

使用 comptime 的泛型函数和类型

```zig
const std = @import("std");

// Generic max function using comptime
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

// Generic struct
fn Pair(comptime T: type) type {
    return struct {
        first: T,
        second: T,

        fn swap(self: *@This()) void {
            const tmp = self.first;
            self.first = self.second;
            self.second = tmp;
        }
    };
}

pub fn main() void {
    // Works with any comparable type
    std.debug.print("max(3, 7) = {}\n", .{max(i32, 3, 7)});
    std.debug.print("max(3.5, 2.1) = {d:.1}\n", .{max(f64, 3.5, 2.1)});

    // Generic struct usage
    var pair = Pair(i32){ .first = 10, .second = 20 };
    std.debug.print("Before swap: {}, {}\n", .{ pair.first, pair.second });
```

```zig
    pair.swap();
    std.debug.print("After swap: {}, {}\n", .{ pair.first, pair.second });
}
```

输出:

```
max(3, 7) = 7
max(3.5, 2.1) = 3.5
Before swap: 10, 20
After swap: 20, 10
```

---

13. 测试

Zig 有内置测试支持。

测试语法

```
┌──────────────────────────────────────────┐
│  test "描述" {                            │
│      try std.testing.expect(true);       │
│      try std.testing.expectEqual(1, 1);  │
│  }                                       │
└──────────────────────────────────────────┘
```

运行测试: `zig test file.zig`

测试工作流:

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│   写测试     │ -> │   zig test   │ -> │    结果      │
└──────────────┘   └──────────────┘   └──────────────┘
                          │
                   ┌──────┴──────┐
                   │             │
                 通过          失败
                   │             │
                 [OK]      [错误信息]
```

示例: **testing**

编写和运行测试

```zig
const std = @import("std");

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn divide(a: i32, b: i32) !i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}
```

```zig
test "add function" {
    try std.testing.expectEqual(add(2, 3), 5);
    try std.testing.expectEqual(add(-1, 1), 0);
}

test "divide function" {
    const result = try divide(10, 2);
    try std.testing.expectEqual(result, 5);
}

test "divide by zero returns error" {
    const result = divide(10, 0);
    try std.testing.expectError(error.DivisionByZero, result);
}

pub fn main() void {
    std.debug.print("Run 'zig test' to execute tests\n", .{});
    std.debug.print("add(2,3) = {}\n", .{add(2, 3)});
}
```

输出:

```
Run 'zig test' to execute tests
add(2,3) = 5
```

---

14. 构建系统

Zig 有一个使用 `build.zig` 的强大内置构建系统。

基本 build.zig

```
┌──────────────────────────────────────┐
│  zig init     - 创建新项目            │
│  zig build    - 构建项目              │
│  zig build run - 构建后运行           │
└──────────────────────────────────────┘
```

项目结构:

```
my_project/
├── build.zig       <- 构建配置
├── build.zig.zon   <- 依赖
└── src/
    ├── main.zig    <- 入口点
    └── lib.zig     <- 库代码
```

构建流程:

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ build.zig    │ → │ zig build    │ → │ zig-out/     │
└──────────────┘   └──────────────┘   └──────────────┘
```

示例: build_example

项目结构和构建命令

```zig
const std = @import("std");

// Example of a library function
pub fn greet(name: []const u8) void {
    std.debug.print("Hello, {s}!\n", .{name});
}

// Example of configuration
pub const Config = struct {
    debug: bool = false,
    optimization: enum { none, speed, size } = .none,
};

pub fn main() void {
    std.debug.print("Build System Demo\n", .{});
    std.debug.print("================\n", .{});
    std.debug.print("\n", .{});
    std.debug.print("Commands:\n", .{});
    std.debug.print("  zig init        - Create new project\n", .{});
    std.debug.print("  zig build       - Build project\n", .{});
    std.debug.print("  zig build run   - Build and run\n", .{});
    std.debug.print("  zig build test  - Run tests\n", .{});
    std.debug.print("\n", .{});
    greet("World");
}
```

输出:

```
Build System Demo
================

Commands:
  zig init        - Create new project
  zig build       - Build project
  zig build run   - Build and run
  zig build test  - Run tests

Hello, World!
```

快速参考

常用命令

```
zig run file.zig     # 编译后运行
zig build-exe file.zig  # 编译为可执行文件
zig test file.zig    # 运行测试
zig fmt file.zig     # 格式化代码
zig init             # 初始化新项目
zig build            # 构建项目
```

有用的内置函数

内置函数	描述
@import	导入模块
@intCast	整数类型转换
@floatFromInt	整数转浮点数
@truncate	截断为更小类型
@sizeOf	获取类型大小
@TypeOf	获取表达式类型

格式说明符

说明符	描述
{}	默认
{s}	字符串
{c}	字符
{d}	十进制
{x}	十六进制
{b}	二进制